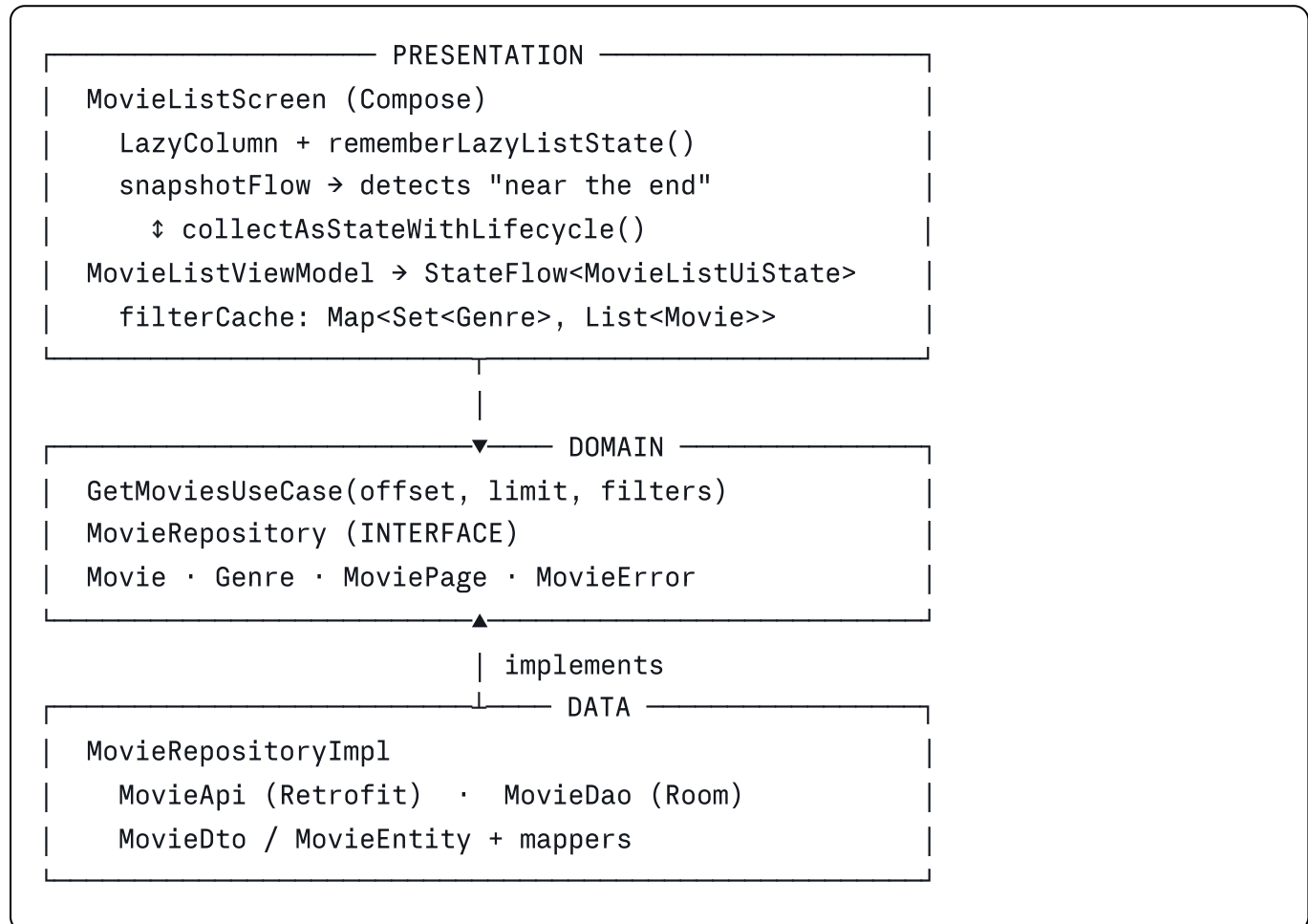


MVVM + Clean Architecture — Movie List (Pagination + Multi-Filter)

1. The 30-Second Interview Opener

"Same three layers as before, but a list screen stresses different things: paginated state, filter combinations, offline caching, and recomposition cost. The ViewModel owns one immutable `MovieListUiState`; the UseCase decides *what* to fetch; the Repository decides *where* it comes from — network or Room. Let me walk through it."

2. Layer Diagram



3. DOMAIN LAYER

3.1 Models

kotlin

```

data class Movie(
    val id: String,           // ALWAYS have a stable id – needed for LazyCo
    val title: String,
    val posterUrl: String,
    val rating: Double,
    val genres: Set<Genre>,
    val isFavourite: Boolean = false
)

enum class Genre { THRILLER, ACTION, ANIMATION, SPORTS, HORROR, DRAMA }

// Wrapping the page tells us "is there more?" without guessing
data class MoviePage(
    val movies: List<Movie>,
    val hasMore: Boolean
)

sealed class MovieError {
    data object NoInternet : MovieError()
    data object ServerError : MovieError()
    data object EmptyResult : MovieError()
    data object Unknown : MovieError()
}

```

Why `MoviePage` instead of a bare `List<Movie>`? Inferring `hasMore = returned.size == pageSize` is a heuristic that breaks when the last page happens to be exactly full. A server-provided flag (or total count) is authoritative. Mention both — say you prefer the explicit flag and fall back to the heuristic if the API doesn't provide one.

3.2 Repository interface

```

kotlin

interface MovieRepository {
    suspend fun getMovies(
        offset: Int,
        limit: Int,
        filters: Set<Genre>
    ): Result<MoviePage>

    suspend fun toggleFavourite(movieId: String, isFavourite: Boolean)
    fun observeFavouriteIds(): Flow<Set<String>>
}

```

3.3 UseCase

kotlin

```
class GetMoviesUseCase @Inject constructor(  
    private val repository: MovieRepository  
) {  
    suspend operator fun invoke(  
        offset: Int,  
        limit: Int,  
        filters: Set<Genre>  
    ): Result<MoviePage> {  
        require(limit in 1..100) { "Page size out of range" }  
        return repository.getMovies(offset, limit, filters)  
    }  
}
```

4. DATA LAYER

4.1 Room entity + DAO

kotlin

```

@Entity(tableName = "movies")
data class MovieEntity(
    @PrimaryKey val id: String,
    val title: String,
    val posterUrl: String,
    val rating: Double,
    val genresCsv: String,
    val isFavourite: Boolean
)

@Dao
interface MovieDao {

    @Query("""
        SELECT * FROM movies
        WHERE (:genreFilter = '' OR genresCsv LIKE '%' || :genreFilter || '%')
        ORDER BY rating DESC
        LIMIT :limit OFFSET :offset
    """)
    suspend fun getMovies(offset: Int, limit: Int, genreFilter: String): List<M

    // REPLACE = upsert. Re-fetching page 3 updates rows in place instead of du
    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun upsertAll(movies: List<MovieEntity>)

    @Query("UPDATE movies SET isFavourite = :fav WHERE id = :id")
    suspend fun setFavourite(id: String, fav: Boolean)

    @Query("SELECT id FROM movies WHERE isFavourite = 1")
    fun observeFavouriteIds(): Flow<List<String>>
}

```

4.2 Repository implementation — network-first with Room fallback

kotlin

```

class MovieRepositoryImpl @Inject constructor(
    private val api: MovieApi,
    private val dao: MovieDao
) : MovieRepository {

    override suspend fun getMovies(
        offset: Int,
        limit: Int,
        filters: Set<Genre>
    ): Result<MoviePage> = withContext(Dispatchers.IO) {
        try {
            val dto = api.getMovies(offset, limit, filters.joinToString(",") {

                // Upsert – never clearAll(). Clearing would wipe rows the UI is
                // currently showing and destroy the user's favourite flags.
                dao.upsertAll(dto.movies.map { it.toEntity() })

                Result.Success(
                    MoviePage(
                        movies = dto.movies.map { it.toDomain() },
                        hasMore = dto.hasMore
                    )
                )
            }
        } catch (e: CancellationException) {
            throw e
        } catch (e: UnknownHostException) {
            // Offline → serve whatever page we cached earlier
            val cached = dao.getMovies(offset, limit, filters.firstOrNull()?.na
            if (cached.isEmpty()) Result.Failure(MovieError.NoInternet)
            else Result.Success(MoviePage(cached.map { it.toDomain() }, hasMore
        } catch (e: HttpException) {
            Result.Failure(
                if (e.code() in 500..599) MovieError.ServerError else MovieError
            )
        }
    }

    override suspend fun toggleFavourite(movieId: String, isFavourite: Boolean) {
        dao.setFavourite(movieId, isFavourite) // Local write is instant
        runCatching { api.setFavourite(movieId, isFavourite) } // Best-effort
    }

    override fun observeFavouriteIds(): Flow<Set<String>> =
        dao.observeFavouriteIds().map { it.toSet() }
}

```

The `clearAll()` trap — say this in the interview: A naive implementation does `dao.clearAll(); dao.insertAll(newPage)`. That deletes the 40 rows the user is currently looking at, drops their local `isFavourite` flags, and makes the list flicker. Always **upsert** with `OnConflictStrategy.REPLACE`.

5. PRESENTATION LAYER

5.1 UI state

kotlin

```
data class MovieListUiState(  
    val movies: List<Movie> = emptyList(),  
    val activeFilters: Set<Genre> = emptySet(),  
    val isInitialLoading: Boolean = false,    // full-screen spinner  
    val isAppending: Boolean = false,         // small spinner at list bottom  
    val hasMore: Boolean = true,  
    val error: String? = null  
) {  
    val isEmpty: Boolean  
        get() = movies.isEmpty() && !isInitialLoading && error == null  
}
```

Two loading flags, not one. First load shows a full-screen spinner; appending page 3 shows a small footer spinner. One boolean would blank the whole screen mid-scroll.

5.2 ViewModel

kotlin

```

@HiltViewModel
class MovieListViewModel @Inject constructor(
    private val getMoviesUseCase: GetMoviesUseCase,
    private val movieRepository: MovieRepository
) : ViewModel() {

    private val _uiState = MutableStateFlow(MovieListUiState())
    val uiState: StateFlow<MovieListUiState> = _uiState.asStateFlow()

    private var loadJob: Job? = null

    // Cache keyed by filter COMBINATION, LRU-capped so memory can't grow unbound
    private val filterCache = object : LinkedHashMap<Set<Genre>, List<Movie>>(
        /* initialCapacity = */ 8, /* loadFactor = */ 0.75f, /* accessOrder = */ true
    ) {
        override fun removeEldestEntry(eldest: Map.Entry<Set<Genre>, List<Movie>>) : Boolean {
            return true
        }
    }

    private companion object { const val PAGE_SIZE = 20 }

    init {
        loadNextPage()
        observeFavourites()
    }

    // ----- PAGINATION -----

    fun loadNextPage() {
        val state = _uiState.value
        if (state.isAppending || state.isInitialLoading || !state.hasMore) return

        val isFirstPage = state.movies.isEmpty()

        loadJob = viewModelScope.launch {
            _uiState.update {
                it.copy(
                    isInitialLoading = isFirstPage,
                    isAppending = !isFirstPage,
                    error = null
                )
            }

            // offset = current size. No separate page counter to drift out of sync
            val offset = _uiState.value.movies.size

            when (val result = getMoviesUseCase(offset, PAGE_SIZE, _uiState.value)) {

```

```

        is Result.Success -> _uiState.update { s ->
            val merged = s.movies + result.data.movies
            filterCache[s.activeFilters] = merged
            s.copy(
                movies = merged,
                hasMore = result.data.hasMore,
                isInitialLoading = false,
                isAppending = false
            )
        }
        is Result.Failure -> _uiState.update {
            it.copy(
                isInitialLoading = false,
                isAppending = false,
                error = result.error.toMessage()
            )
        }
    }
}

fun retry() = loadNextPage()

// ----- MULTI-FILTER -----

fun toggleFilter(genre: Genre) {
    val current = _uiState.value.activeFilters
    val newFilters = if (genre in current) current - genre else current + genre
    applyFilters(newFilters)
}

fun clearFilters() = applyFilters(emptySet())

private fun applyFilters(newFilters: Set<Genre>) {
    // In-flight request belongs to the OLD filter – cancel it or its result
    // will be appended to the wrong list.
    loadJob?.cancel()

    // Snapshot the outgoing list so switching back is instant
    filterCache[_uiState.value.activeFilters] = _uiState.value.movies

    val cached = filterCache[newFilters]

    if (cached != null) {
        _uiState.update {
            it.copy(movies = cached, activeFilters = newFilters, error = null)
        }
    }
}

```



```

    } else {
        _uiState.update {
            MovieListUiState(activeFilters = newFilters) // fresh state, e
        }
        loadNextPage()
    }
}

// ----- FAVOURITES -----

fun toggleFavourite(movie: Movie) {
    viewModelScope.launch {
        // Optimistic update – UI flips instantly, no waiting on the network
        _uiState.update { s ->
            s.copy(movies = s.movies.map {
                if (it.id == movie.id) it.copy(isFavourite = !it.isFavourite)
            })
        }
        movieRepository.toggleFavourite(movie.id, !movie.isFavourite)
    }
}

private fun observeFavourites() {
    viewModelScope.launch {
        movieRepository.observeFavouriteIds().collect { favIds ->
            _uiState.update { s ->
                s.copy(movies = s.movies.map { it.copy(isFavourite = it.id
                    in favIds) })
            }
        }
    }
}

private fun MovieError.toMessage(): String = when (this) {
    MovieError.NoInternet -> "No internet connection"
    MovieError.ServerError -> "Something went wrong. Please try again"
    MovieError.EmptyResult -> "No movies match these filters"
    MovieError.Unknown -> "Unexpected error. Please try again"
}
}

```

5.3 Compose UI — pagination trigger

kotlin

@Composable

```
fun MovieListScreen(viewModel: MovieListViewModel = hiltViewModel()) {

    val state by viewModel.uiState.collectAsStateWithLifecycle()
    val listState = rememberLazyListState()

    // Detect "user is near the end" and request the next page.
    // distinctUntilChanged() stops this firing on every pixel of scroll.
    LaunchedEffect(listState, state.activeFilters) {
        snapshotFlow {
            val lastVisible = listState.layoutInfo.visibleItemsInfo.lastOrNull()
            val total = listState.layoutInfo.totalItemsCount
            total > 0 && lastVisible >= total - 5 // prefetch threshold
        }
        .distinctUntilChanged()
        .filter { it }
        .collect { viewModel.loadNextPage() }
    }

    Column {
        FilterChipRow(
            active = state.activeFilters,
            onToggle = viewModel::toggleFilter,
            onClear = viewModel::clearFilters
        )

        LazyColumn(
            state = listState,
            modifier = Modifier.fillMaxSize(),
            contentPadding = PaddingValues(16.dp),
            verticalArrangement = Arrangement.spacedBy(12.dp)
        ) {
            items(
                items = state.movies,
                key = { it.id } // stable identity - critical
            ) { movie ->
                MovieCard(
                    movie = movie,
                    onFavouriteClick = { viewModel.toggleFavourite(movie) }
                )
            }

            if (state.isAppending) {
                item(key = "footer_loader") {
                    Box(
                        Modifier.fillMaxWidth().padding(16.dp),

```

```

        contentAlignment = Alignment.Center
    ) { CircularProgressIndicator() }
    }
}

state.error?.let { message ->
    item(key = "footer_error") {
        ErrorFooter(message = message, onRetry = viewModel::retry)
    }
}
}
}
}
}
}
}
}
}

```

6. What Each Piece Buys You

Technique	Problem it solves
<code>key = { it.id }</code> in <code>items()</code>	Without it, scroll causes items to recompose and item-local state (checkbox, expanded) resets or jumps to the wrong row.
<code>rememberLazyListState()</code>	Survives recomposition so scroll position isn't lost.
<code>snapshotFlow { ... }</code>	Turns Compose state (<code>layoutInfo</code>) into a Flow so you can <i>react</i> to scroll instead of polling.
<code>distinctUntilChanged()</code>	Without it, the threshold check emits on every frame → dozens of duplicate page requests.
Prefetch threshold (<code>(total - 5)</code>)	Loads the next page <i>before</i> the user hits the bottom → no visible stall.
<code>offset = movies.size</code>	Self-correcting. A separate page counter drifts if a request fails or is cancelled.
<code>@Immutable</code> on <code>Movie</code>	Tells Compose the type is stable → it can skip recomposing unchanged rows.
Two loading flags	First-load spinner vs. append spinner — one boolean would blank the screen mid-scroll.

7. EDGE CASES — the probing questions

Edge Case 1 — Duplicate page requests during fast scroll

`snapshotFlow` fires on every frame while scrolling. Guards:

1. `distinctUntilChanged()` in the Flow.

2. `if (state.isAppending) return` at the top of `loadNextPage()`. Without both, a fast flick fires 30+ identical requests.

Edge Case 2 — Filter changes while a page request is in flight

The in-flight request was issued for the *old* filter. If it lands after the state reset, thriller results get appended to a sports list. **Fix:** `loadJob?.cancel()` at the start of `applyFilters()`. This is the single most commonly missed bug in list screens.

Edge Case 3 — Server data changed between page loads

User loads pages 1-2 (40 movies). New movies are inserted server-side. Offset 40 now points at a shifted window → duplicates or skipped items.

- **Mitigation:** cursor/keyset pagination (`?after=<lastMovieId>`) instead of offset. Offset pagination is inherently unstable on mutating datasets.
- **Room-side mitigation:** `OnConflictStrategy.REPLACE` upserts, so duplicates collapse into one row instead of appearing twice.

Edge Case 4 — Clearing vs. keeping cached results on filter change

This is a genuine trade-off, and interviewers like hearing it framed as one:

- **Clear + reload:** simple, low memory, but the user loses scroll position and re-pays the network cost.
- **Cache per filter combination:** instant switching, preserves work, but memory grows with each combination.
- **My answer:** cache keyed by the filter (`Set`), LRU-capped at ~5 entries. Cheap combinations stay warm; memory stays bounded. Switching back to a previously viewed filter is instant.

Edge Case 5 — Filtered result smaller than one page

User filters to a genre with 10 movies. Scroll hits bottom, `loadNextPage()` fires, server returns `hasMore = false`. State sets `hasMore = false` → the guard blocks all further requests. No infinite loop. Show an "end of list" footer rather than a spinner.

Edge Case 6 — Item state lost when scrolling off-screen

`remember { mutableStateOf(false) }` **inside** the item composable is destroyed when the item leaves the viewport. An expanded card collapses; a checkbox unchecks. **Fix — hoist to the parent, keyed by id:**

```
kotlin
```

```
// In ViewModel state, not inside the item
val expandedIds: Set<String> = emptySet()

MovieCard(
    isExpanded = movie.id in state.expandedIds,
    onToggle = { viewModel.toggleExpanded(movie.id) }
)
```

Edge Case 7 — Configuration change mid-scroll

ViewModel survives → the loaded list survives. `rememberLazyListState()` uses `rememberSaveable` internally, so scroll position is restored too. But an in-flight `viewModelScope` coroutine keeps running — correct behaviour, since the ViewModel outlives the Activity.

Edge Case 8 — Offline

Repository catches `UnknownHostException`, serves the cached page from Room, and returns `hasMore = false` so pagination stops cleanly instead of retrying into a wall. If the cache is empty too, surface `NoInternet` with a retry button.

8. Testability

```
kotlin

class MovieListViewModelTest {

    @Test
    fun `filter change cancels in-flight request and resets list`() = runTest {
        val useCase = mockk<GetMoviesUseCase>()
        coEvery { useCase(any(), any(), emptySet()) } returns
            Result.Success(MoviePage(fakeMovies(20), hasMore = true))
        coEvery { useCase(any(), any(), setOf(Genre.THRILLER)) } returns
            Result.Success(MoviePage(fakeMovies(5), hasMore = false))

        val vm = MovieListViewModel(useCase, mockk(relaxed = true))
        advanceUntilIdle()
        assertEquals(20, vm.uiState.value.movies.size)

        vm.toggleFilter(Genre.THRILLER)
        advanceUntilIdle()

        assertEquals(5, vm.uiState.value.movies.size) // old results discarded
        assertFalse(vm.uiState.value.hasMore)
    }
}
```

9. Rapid-Fire Answers

Question	Answer
Why not just use Paging 3?	For production, I would — it handles cursors, retries, and <code>RemoteMediator</code> for network+DB. I'm hand-rolling here to show the mechanics. Mention Paging 3 by name; interviewers expect it.
How does the ViewModel know the user scrolled?	It doesn't observe scroll directly. The UI watches <code>LazyListState.layoutInfo</code> via <code>snapshotFlow</code> and calls <code>loadNextPage()</code> . Keeps the ViewModel free of UI-layout concerns.
Why <code>offset = movies.size</code> instead of a page counter?	Self-correcting. A counter drifts if a request fails, is cancelled, or a filter resets mid-flight.
Offset vs cursor pagination?	Offset is simple but unstable when the dataset mutates. Cursor/keyset is stable and is what I'd use against a live catalogue.
Why does <code>key()</code> matter so much?	It's how Compose maps data to composable instances. Without it, position is the identity — insert one item at the top and every row below re-renders with the wrong state.
Where do favourites live?	Room is the source of truth (<code>observeFavouriteIds()</code> Flow). UI updates optimistically for instant feedback; the network sync is best-effort.
How do you avoid recomposing the whole list?	Stable <code>key()</code> , <code>@Immutable</code> domain models, and appending to the list rather than replacing it. Compose then only composes the new items.
What if the API has no <code>hasMore</code> field?	Fall back to <code>returned.size == pageSize</code> . Note the flaw out loud: a final page that's exactly full triggers one wasted empty request.